Name: Varshitha Darisa

Reg no:24011101024

Roll no:24110098

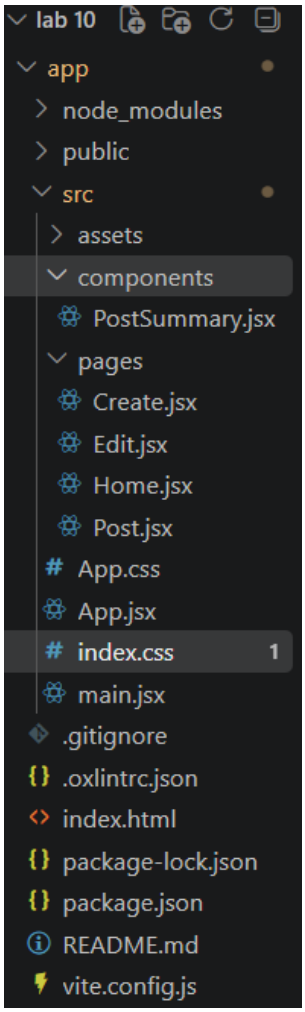
Class : AIDS-'A'

Web Technologies Lab 10

Aim: Develop a small full-stack web application in which users can create, view, update, and delete blog posts. The application must use React for the front end, Express.js and Node.js for the REST API, and MongoDB for storing the post data.

Code: Frontend

Folder structure for frontend



1. **App.jsx**

```jsx
import { useState } from "react";
import "./App.css";

import Home from "./pages/Home";
import Create from "./pages/Create";
import Post from "./pages/Post";
import Edit from "./pages/Edit";

function App() {

    const [page, setPage] = useState("home");

    const [selectedPostId, setSelectedPostId] = useState(null);

    const handleViewPost = (id) => {

        setSelectedPostId(id);
        setPage("post");

    };

    const handleEditPost = (id) => {

        setSelectedPostId(id);
        setPage("edit");

    };

    const handleDeletePost = async (id) => {

        const confirmDelete = window.confirm(
            "Are you sure you want to delete this post?"
        );

        if (!confirmDelete) {
            return;
        }

        try {

            const response = await fetch(
                `http://localhost:3000/api/posts/${id}`,
                {
                    method: "DELETE"
                }
            );

```

```
            const data = await response.json();

            if (!response.ok) {

                alert(
                    data.message || "Failed to delete post"
                );

                return;
            }

            alert("Post deleted successfully");

            window.location.reload();

        } catch (error) {

            console.error(error);

            alert("Unable to connect to the server");

        }
    };

    return (
        <div className="app">

            {page === "home" && (
                <>
                    <div className="app-header">

                        <h1>My Blog</h1>

                        <button
                            className="primary-btn"
                            onClick={() => setPage("create")}
                        >
                            + Create New Post
                        </button>

                    </div>

                    <Home
                        onView={handleViewPost}
                        onEdit={handleEditPost}
                        onDelete={handleDeletePost}
                    />
                </>
```

```
            )}

            {page === "create" && (
                    <Create
                        onPostCreated={() => setPage("home")}
                        onCancel={() => setPage("home")}
                    />
                )}

                {page === "post" && (
                    <Post
                        postId={selectedPostId}
                        onBack={() => setPage("home")}
                    />
                )}

                {page === "edit" && (
                    <Edit
                        postId={selectedPostId}
                        onUpdated={() => setPage("home")}
                        onCancel={() => setPage("home")}
                    />
                )}

            </div>
        );
    }

    export default App;
```

2. App.css

```
* {
    box-sizing: border-box;
}

/* ---------- BODY ---------- */

body {
    margin: 0;
    font-family: Arial, sans-serif;
    background-color: #f5f7fb;
    color: #222;
}

#root {
    min-height: 100vh;
}
```

```css
/* ---------- MAIN APP ---------- */

.app {
    width: 90%;
    max-width: 1100px;
    margin: 0 auto;
    padding: 35px 0;
}

/* ---------- HEADER ---------- */

.app-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 30px;
}

.app-header h1 {
    margin: 0;
    font-size: 32px;
}

/* ---------- GENERAL BUTTON ---------- */

button {
    border: none;
    padding: 10px 16px;
    border-radius: 6px;
    font-size: 14px;
    cursor: pointer;
    transition: 0.2s ease;
}

button:hover {
    opacity: 0.85;
}

/* ---------- BUTTON TYPES ---------- */

.primary-btn {
    background-color: #2563eb;
    color: white;
}

.secondary-btn {
    background-color: #e5e7eb;
```

```
    color: #222;
}

.view-btn {
    background-color: #2563eb;
    color: white;
}

.edit-btn {
    background-color: #f59e0b;
    color: white;
}

.delete-btn {
    background-color: #dc2626;
    color: white;
}

/* ---------- POSTS CONTAINER ---------- */

.posts {
    display: grid;
    grid-template-columns: repeat(2, 1fr);
    gap: 22px;
}

/* ---------- POST CARD ---------- */

.post {
    background-color: white;
    padding: 24px;
    border-radius: 10px;
    border: 1px solid #ddd;
    box-shadow: 0 3px 8px rgba(0, 0, 0, 0.05);
}

.post h2 {
    margin-top: 0;
    margin-bottom: 12px;
    font-size: 22px;
}

.post-content {
    line-height: 1.6;
    color: #555;
}

.post-author {
```

```
    font-size: 14px;
    color: #444;
}

.post-date {
    font-size: 12px;
    color: #777;
}

/* ---------- POST BUTTONS ---------- */

.post-actions {
    display: flex;
    gap: 10px;
    margin-top: 20px;
}

/* ---------- CREATE / EDIT PAGE ---------- */

.create-page {
    background-color: white;
    padding: 35px;
    border-radius: 10px;
    border: 1px solid #ddd;
    max-width: 700px;
    margin: 30px auto;
    box-shadow: 0 3px 8px rgba(0, 0, 0, 0.05);
}

.create-page h1 {
    margin-top: 0;
    margin-bottom: 25px;
}

/* ---------- FORM ---------- */

.form-group {
    margin-bottom: 20px;
}

.form-group label {
    display: block;
    margin-bottom: 8px;
    font-weight: bold;
}

.form-group input,
.form-group textarea {
```

```
    width: 100%;
    padding: 12px;
    border: 1px solid #ccc;
    border-radius: 6px;
    font-size: 15px;
    font-family: Arial, sans-serif;
}

.form-group input:focus,
.form-group textarea:focus {
    outline: none;
    border-color: #2563eb;
}

.form-group textarea {
    resize: vertical;
}

/* ---------- FORM BUTTONS ---------- */

.create-page button {
    margin-right: 10px;
}

/* ---------- SINGLE POST PAGE ---------- */

.single-post {
    background-color: white;
    padding: 35px;
    border-radius: 10px;
    border: 1px solid #ddd;
    margin-top: 30px;
    box-shadow: 0 3px 8px rgba(0, 0, 0, 0.05);
}

.single-post h1 {
    margin-top: 25px;
    margin-bottom: 20px;
    font-size: 32px;
}

.single-post .post-content {
    line-height: 1.8;
    font-size: 18px;
    margin: 25px 0;
    color: #444;
}
```

```css
/* ---------- MESSAGES ---------- */

.success-message {
    color: green;
    margin-top: 20px;
}

.error-message {
    color: red;
    margin-top: 20px;
}

/* ---------- EMPTY POSTS ---------- */

.empty-message {
    background-color: white;
    padding: 40px;
    text-align: center;
    border-radius: 10px;
    border: 1px solid #ddd;
    color: #777;
}

/* ---------- MOBILE ---------- */

@media (max-width: 700px) {

    .app {
        width: 94%;
        padding: 25px 0;
    }

    .app-header {
        flex-direction: column;
        align-items: flex-start;
        gap: 15px;
    }

    .posts {
        grid-template-columns: 1fr;
    }

    .post-actions {
        flex-wrap: wrap;
    }

    .create-page,
    .single-post {
```

```
        padding: 22px;
    }

    .single-post h1 {
        font-size: 26px;
    }
}
```

3. components/PostSummary.jsx

```
function PostSummary({ post, onView, onEdit, onDelete }) {
    return (
        <article className="post">

            <h2>{post.title}</h2>

            <p className="post-content">
                {post.content}
            </p>

            <p className="post-author">
                <strong>Author:</strong> {post.author}
            </p>

            <p className="post-date">
                Created:{" "}
                {new Date(post.createdAt).toLocaleString()}
            </p>

            <div className="post-actions">

                <button
                    className="view-btn"
                    onClick={() => onView(post._id)}
                >
                    View
                </button>

                <button
                    className="edit-btn"
                    onClick={() => onEdit(post._id)}
                >
                    Edit
                </button>

                <button
```

```
                    className="delete-btn"
                    onClick={() => onDelete(post._id)}
                >
                    Delete
                </button>

            </div>

        </article>
    );
}

export default PostSummary;
```

4. pages/Create.jsx

```
import { useState } from "react";

function Create({ onPostCreated, onCancel }) {
    const [title, setTitle] = useState("");
    const [author, setAuthor] = useState("");
    const [content, setContent] = useState("");

    const [message, setMessage] = useState("");
    const [error, setError] = useState("");

    const handleSubmit = async (event) => {
        event.preventDefault();

        setMessage("");
        setError("");

        try {
            const response = await fetch("http://localhost:3000/api/posts", {
                method: "POST",
                headers: {
                    "Content-Type": "application/json"
                },
                body: JSON.stringify({
                    title,
                    author,
                    content
                })
            });

            const data = await response.json();

            if (!response.ok) {
```

```
            setError(data.message || "Failed to create post");
            return;
        }

        setMessage("Post created successfully!");

        setTitle("");
        setAuthor("");
        setContent("");

        if (onPostCreated) {
            onPostCreated(data.post);
        }

    } catch (error) {
        console.error(error);
        setError("Unable to connect to the server");
    }
};

return (
    <div className="create-page">

        <h1>Create New Post</h1>

        <form onSubmit={handleSubmit}>

            <div className="form-group">
                <label>Title</label>

                <input
                    type="text"
                    value={title}
                    onChange={(event) => setTitle(event.target.value)}
                    placeholder="Enter post title"
                />
            </div>

            <div className="form-group">
                <label>Author</label>

                <input
                    type="text"
                    value={author}
                    onChange={(event) => setAuthor(event.target.value)}
                    placeholder="Enter author name"
                />
            </div>
```

```
                <div className="form-group">
                    <label>Content</label>

                    <textarea
                        value={content}
                        onChange={(event) => setContent(event.target.value)}
                        placeholder="Write your post..."
                        rows="8"
                    />
                </div>

                <button type="submit">
                    Create Post
                </button>

                <button
                    type="button"
                    onClick={onCancel}
                >
                    Cancel
                </button>

            </form>

            {message && (
                <p className="success-message">
                    {message}
                </p>
            )}

            {error && (
                <p className="error-message">
                    {error}
                </p>
            )}

        </div>
    );
}

export default Create;
```

4. pages/Edit.jsx

```
import { useEffect, useState } from "react";

function Edit({ postId, onUpdated, onCancel }) {
```

```
  const [title, setTitle] = useState("");
  const [author, setAuthor] = useState("");
  const [content, setContent] = useState("");

  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  // Load the existing post
  useEffect(() => {

      const fetchPost = async () => {

          try {

              const response = await fetch(
                  `http://localhost:3000/api/posts/${postId}`
              );

              const data = await response.json();

              if (!response.ok) {
                  setError(data.message || "Failed to load post");
                  return;
              }

              setTitle(data.title);
              setAuthor(data.author);
              setContent(data.content);

          } catch (error) {

              console.error(error);
              setError("Unable to connect to the server");

          } finally {

              setLoading(false);

          }
      };

      fetchPost();

  }, [postId]);


  // Update the post
  const handleSubmit = async (event) => {
```

```
    event.preventDefault();

    setError("");

    try {

        const response = await fetch(
            `http://localhost:3000/api/posts/${postId}`,
            {
                method: "PUT",

                headers: {
                    "Content-Type": "application/json"
                },

                body: JSON.stringify({
                    title,
                    author,
                    content
                })
            }
        );

        const data = await response.json();

        if (!response.ok) {
            setError(data.message || "Failed to update post");
            return;
        }

        onUpdated();

    } catch (error) {

        console.error(error);
        setError("Unable to connect to the server");

    }
};


if (loading) {
    return <p>Loading post...</p>;
}


return (
    <div className="create-page">
```

```jsx
<h1>Edit Post</h1>

{error && (
    <p className="error-message">
        {error}
    </p>
)}

<form onSubmit={handleSubmit}>

    <div className="form-group">

        <label>Title</label>

        <input
            type="text"
            value={title}
            onChange={(event) =>
                setTitle(event.target.value)
            }
        />

    </div>


    <div className="form-group">

        <label>Author</label>

        <input
            type="text"
            value={author}
            onChange={(event) =>
                setAuthor(event.target.value)
            }
        />

    </div>


    <div className="form-group">

        <label>Content</label>

        <textarea
            rows="8"
            value={content}
            onChange={(event) =>
```

```
                            setContent(event.target.value)
                        }
                    />

                </div>


                <button
                    type="submit"
                    className="primary-btn"
                >
                    Update Post
                </button>


                <button
                    type="button"
                    className="secondary-btn"
                    onClick={onCancel}
                >
                    Cancel
                </button>

            </form>

        </div>
    );
}

export default Edit;
```

4. pages/Home.jsx

```
import {useState,useEffect} from "react";
import PostSummary from "../components/PostSummary";

function Home({ onView, onEdit, onDelete }){
    const [posts,setPosts] = useState([]);
    const [loading,setLoading] = useState(true);
    const [error,setError] = useState("");

    useEffect(()=>{
        fetchPosts();
    },[]);

    async function fetchPosts(){
        try{
            const response = await fetch(
                "http://localhost:3000/api/posts"
```

```
            );

            if(!response.ok){
                throw new Error("Failed to fetch posts");
            }
            const data = await response.json();

            setPosts(data);
        }
        catch(e){
            console.error(e.message);
            setError("Unable to load the posts");
        }
        finally{
            setLoading(false);
        }
    }
    if(loading){
        return <h2>Loading posts...</h2>
    }
    if(error){
        return <h2>{error}</h2>
    }

   return (
    <div>
        <h1>Blog Posts</h1>

        {posts.length === 0 ? (
            <p>No posts available.</p>
        ) : (
            <div className="posts">
                {posts.map((post) => (
                    <PostSummary
    key={post._id}
    post={post}
    onView={onView}
    onEdit={onEdit}
    onDelete={onDelete}
/>
                ))}
            </div>
        )}
    </div>
);
}
export default Home;
```

5. pages/Post.jsx

```jsx
import { useEffect, useState } from "react";

function Post({ postId, onBack }) {
    const [post, setPost] = useState(null);
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState("");

    useEffect(() => {
        const fetchPost = async () => {
            try {
                const response = await fetch(
                    `http://localhost:3000/api/posts/${postId}`
                );

                const data = await response.json();

                if (!response.ok) {
                    setError(data.message || "Failed to fetch post");
                    return;
                }

                setPost(data);

            } catch (error) {
                console.error(error);
                setError("Unable to connect to the server");
            } finally {
                setLoading(false);
            }
        };

        fetchPost();
    }, [postId]);

    if (loading) {
        return <p>Loading post...</p>;
    }

    if (error) {
        return (
            <div>
                <p className="error-message">{error}</p>

                <button onClick={onBack}>
                    Back to Posts
                </button>
            </div>
```

```
        );
    }

    return (
        <div className="single-post">

            <button onClick={onBack}>
                ← Back to Posts
            </button>

            <h1>{post.title}</h1>

            <p className="post-content">
                {post.content}
            </p>

            <p className="post-author">
                <strong>Author:</strong> {post.author}
            </p>

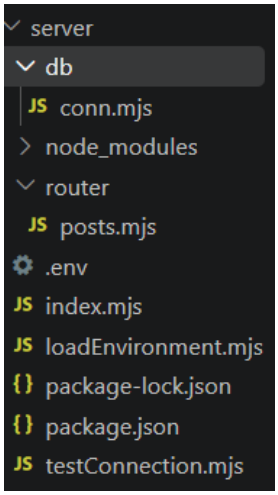
            <p className="post-date">
                Created:{" "}
                {new Date(post.createdAt).toLocaleString()}
            </p>

        </div>
    );
}

export default Post;
```

Code:Backend

Folder structure



1. **db/conn.mjs**

```
import {MongoClient} from "mongodb";
const uri=process.env.MONGODB_URI;
const client = new MongoClient(uri);
let db; // database object
export async function connectToDatabase(){
    if(db){
        return db;
    }
    await client.connect();// connects to the mongodb
    db = client.db("blogDB");// selects the database which we want to work on
    console.log("The database is connected successfully");
    return db;
}
```

2. routes/post.mjs

```
import {Router} from "express";
import {ObjectId} from "mongodb";

import {connectToDatabase} from "../db/conn.mjs";
const router = Router();

router.get("/", async(req,res)=>{
    try{
    const db = await connectToDatabase();
    const posts = await db.collection("posts")
    .find().sort({createdAt:-1}).toArray();
    // -1 means descending order, 1 means ascending order
    res.status(200).json(posts); // send is executed immediately
    }catch(e){
        console.error(e);
        res.status(500).json({message:"Failed to fetch posts"});
    }
})


router.get('/:id',async(req,res)=>{
    try{
        const db = await connectToDatabase();
        const {id}= req.params;
        if(!ObjectId.isValid(id)){
            return res.status(400).json({
                message:"Invalid post ID"
            });
```

```
        }
        const post = await db.collection("posts")
        .findOne({_id:new ObjectId(id)});
        if(!post){
            return res.status(404).json({message:"Post not found"});
        }
        res.status(200).json(post);

    }catch(e){
        console.error(e);
        res.status(500).json({message:"Failed to fetch post"});
    }
});

router.post('/',async(req,res)=>{
    try{
        const db = await connectToDatabase();
        const {title,author,content}=req.body;
        if(!title||!content||!author){
            return res.status(400).json({
                message:"Title, Author and Content are required"
            });
        }
        const newPost = {
            title,
            content,
            author,
            createdAt:new Date(),
            updatedAt:new Date()
        };
        const result = await db
        .collection("posts")
        .insertOne(newPost);

        res.status(201).json({
            message:"The post is created successfully",
            post:{
                _id:result.insertedId,
                ...newPost
            }
        });
    }catch(e){
        console.error(e);
        res.status(500).json({
            message:"Failed to create the post"
        });
    }
});
```

```
router.put('/:id',async(req,res)=>{
    try{
        const db = await connectToDatabase();
        const {id}= req.params;
        const {title,author,content}= req.body;

        if(!ObjectId.isValid(id)){
            return res.status(400).json({
                message:"Invalid post ID"
            });
        }
        if(!title||!author||!content){
            return res.status(400).json({
                message:"Title, Author and Content are required"
            })
        }
        const result = await db
        .collection("posts")
        .updateOne({_id:new ObjectId(id)},
        {$set:{
            title,
            content,
            author,
            updatedAt:new Date()
        }}
    );
    if(result.matchedCount===0){
        return res.status(404).json({
            message:"Post not found"
        });
    }
    res.status(200).json({
        message:"Post updated successfully"
    });
    }catch(e){
        console.error(e);
        res.status(500).json({
            message:"Failed to update the post"
        });
    }
});

router.delete("/:id",async(req,res)=>{
    try{
        const db = await connectToDatabase();
        const {id}=req.params;
        if(!ObjectId.isValid(id)){
```

```
            return res.status(400).json({
                message:"Invalid post ID"
            });
        }

        const result = await db.collection("posts")
        .deleteOne({_id:new ObjectId(id)});

        if(result.deletedCount===0){
             return res.status(404).json({
                message:"Post not found"
            });
        }
        res.status(200).json({
            message:"post deleted successfully"
        });

    }catch(e){
         console.error(e);
        res.status(500).json({
            message:"Failed to Delete the post"
        });
    }
});


export default router;
```

3. index .mjs

```
import './loadEnvironment.mjs';
import express from  "express";
import cors from "cors";
import {connectToDatabase} from "./db/conn.mjs";

import postRouters from "./router/posts.mjs";

const app = express();
app.use(cors()); //middleware
app.use(express.json()); // middleware

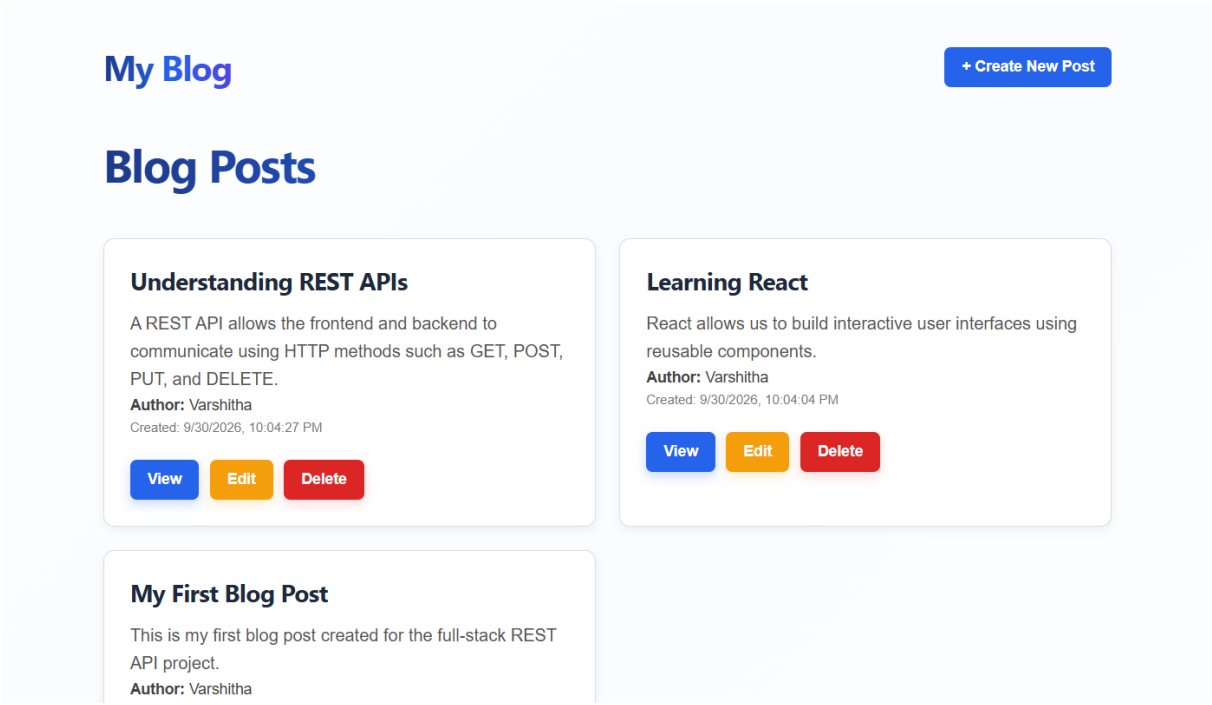
app.use("/api/posts",postRouters);
const PORT = process.env.PORT||3000;
try{
    await connectToDatabase();
    app.listen(PORT,()=>{
        console.log("The application is running successfully on port",PORT);
```

```
    });
}catch(e){
    console.error("Failed to start the server");
    console.error(e.message);
}
```

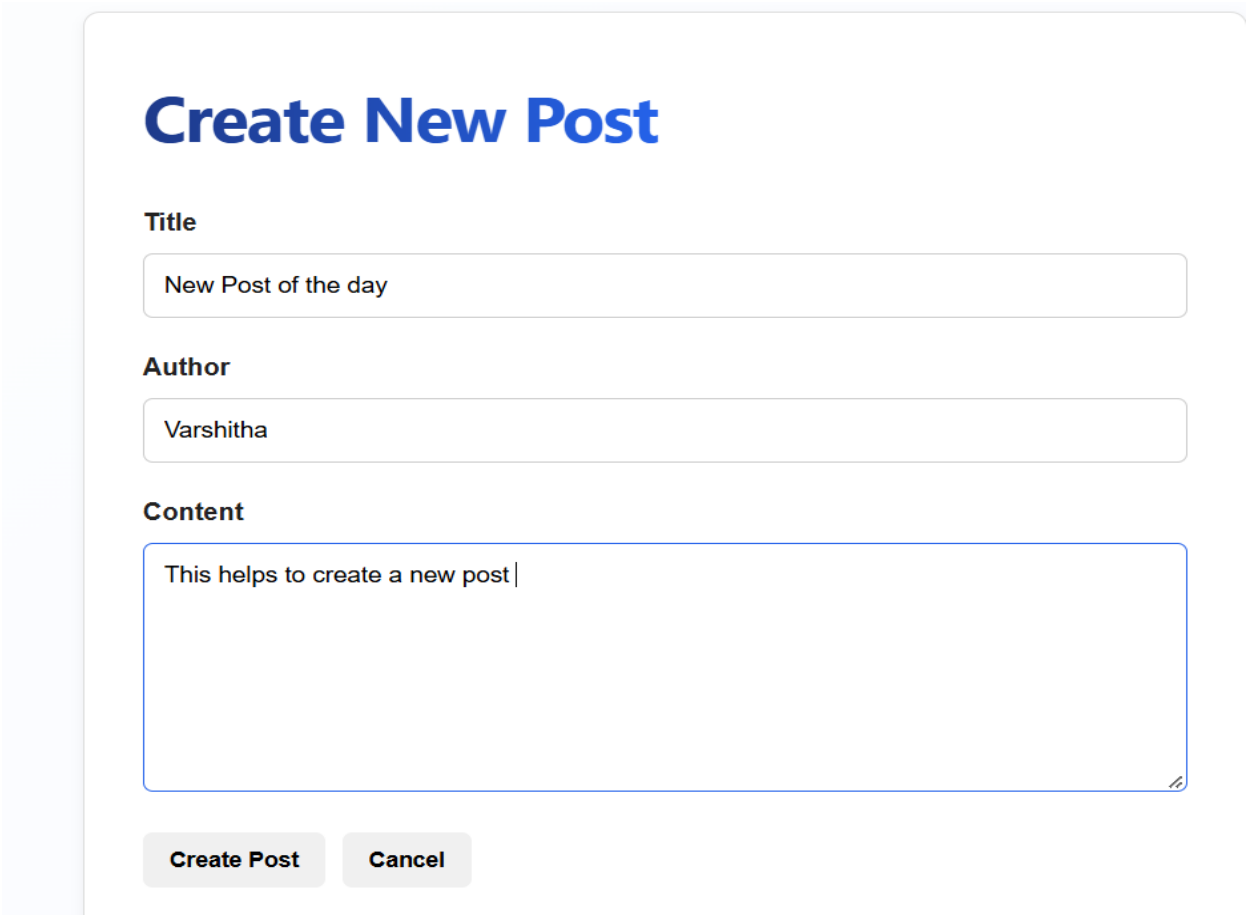
4. loadEnviroment.mjs

```
import dotenv from "dotenv";
dotenv.config();
```

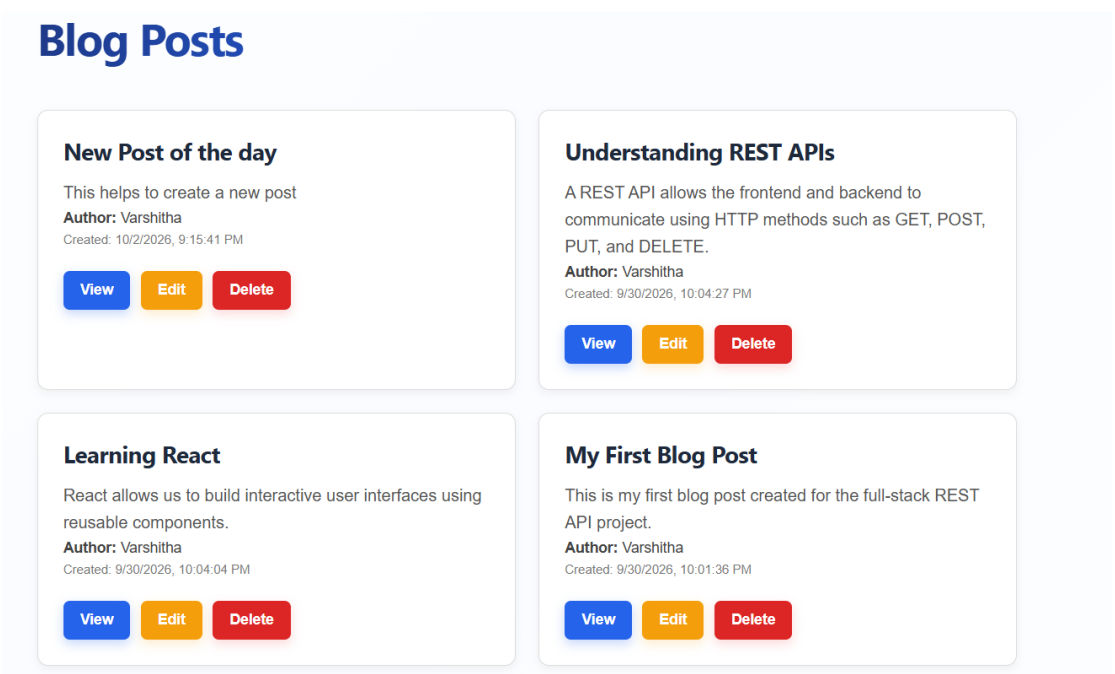
Screenshot of the website:



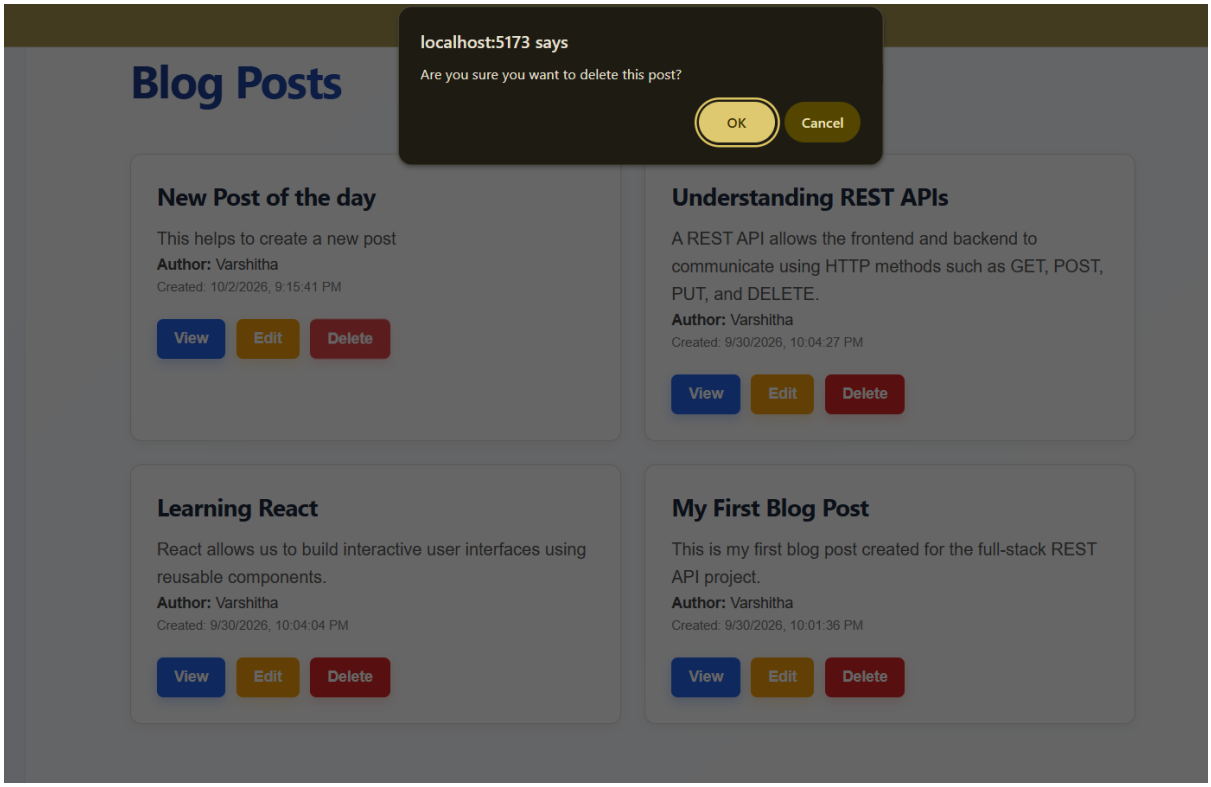
Create a new post



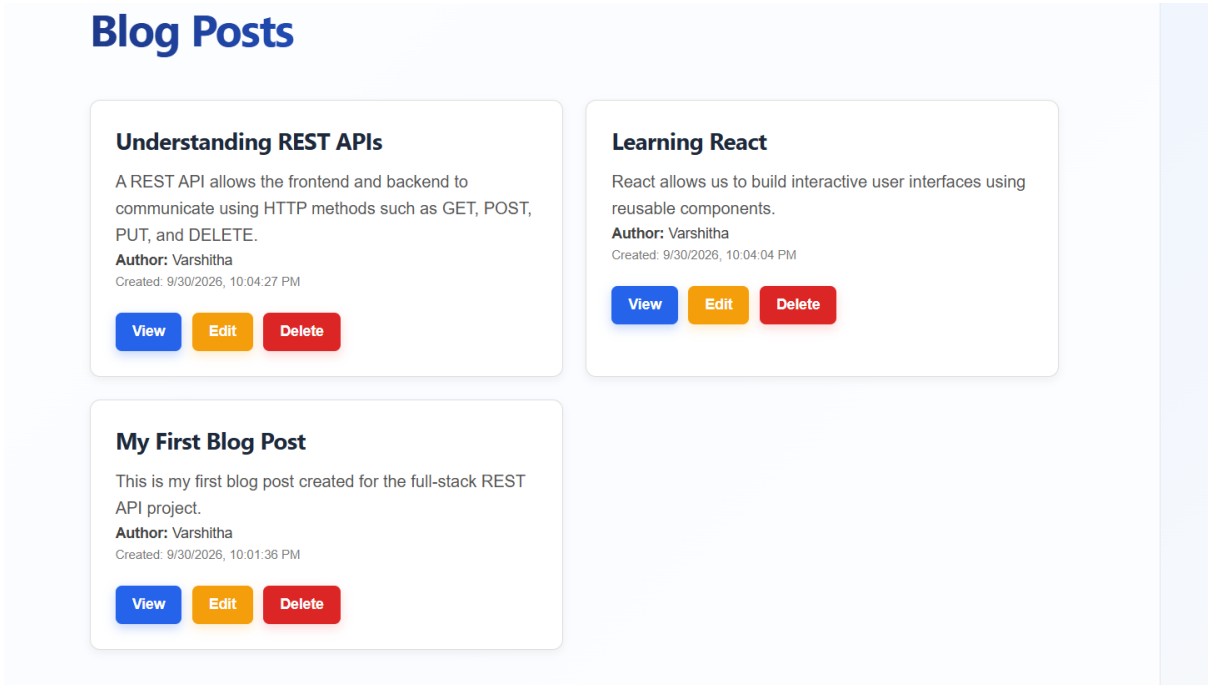
New post is added in the page



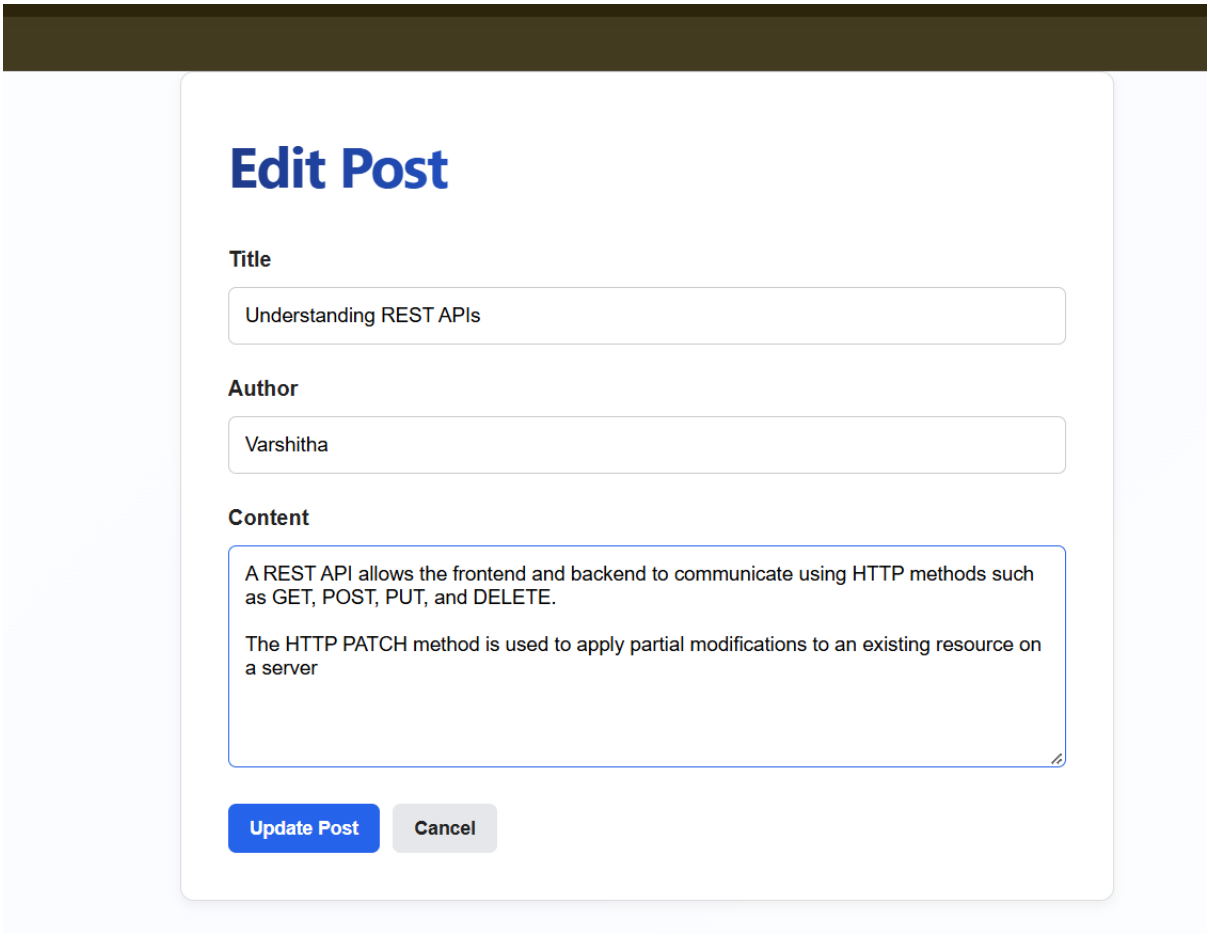
Deleting the post using the delete button

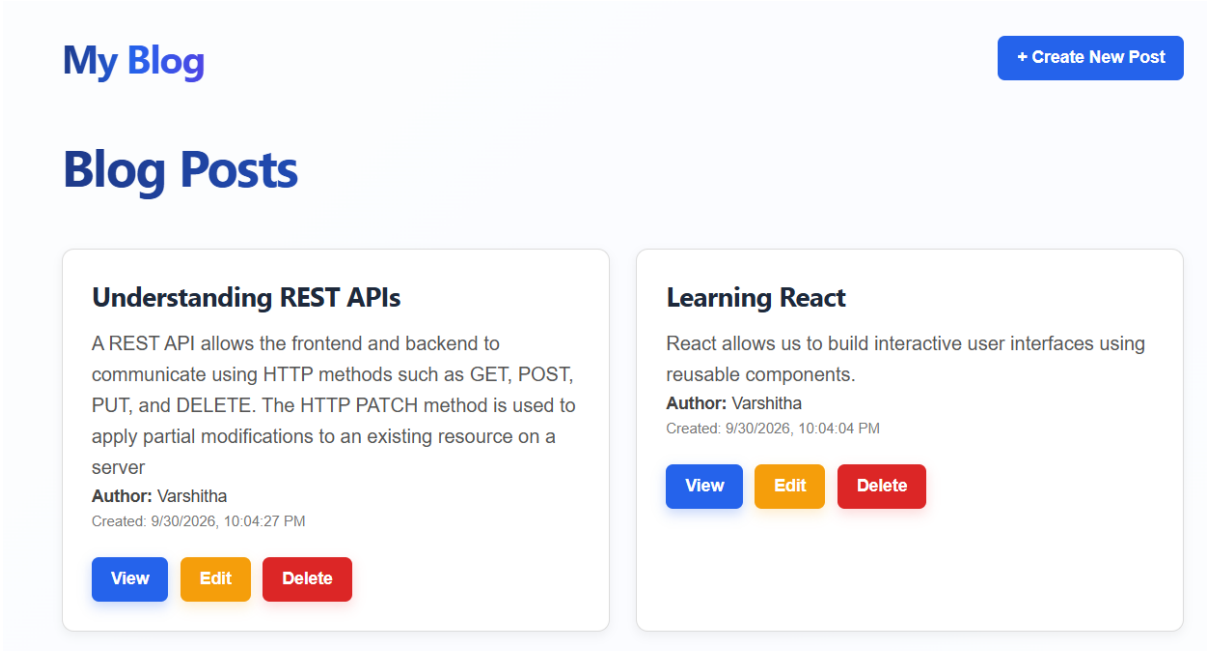


Page after deleting the post



1. Editing the added task


Edit Post
Title
Understanding REST APIs
Author
Varshitha
Content
A REST API allows the frontend and backend to communicate using HTTP methods such as GET, POST, PUT, and DELETE.
The HTTP PATCH method is used to apply partial modifications to an existing resource on a server
Update Post
Cancel


My Blog
+ Create New Post
Blog Posts
Understanding REST APIs
A REST API allows the frontend and backend to communicate using HTTP methods such as GET, POST, PUT, and DELETE. The HTTP PATCH method is used to apply partial modifications to an existing resource on a server
Author: Varshitha
Created: 9/30/2026, 10:04:27 PM
View
Edit
Delete
Learning React
React allows us to build interactive user interfaces using reusable components.
Author: Varshitha
Created: 9/30/2026, 10:04:04 PM
View
Edit
Delete